Scenario: Comparing Student Marks With and Without NumPy

1. Scenario Description

A college wants to calculate the total and average marks of 1,000 students across five subjects. The college also wants to identify the highest and lowest marks and find students who scored above 80.

We can perform these calculations using normal Python lists and loops, without NumPy. However, as the dataset grows, the code can become longer and require more manual operations.

By using NumPy arrays and built-in mathematical functions, we can perform the same calculations with fewer lines of code and more convenient operations on entire datasets.

2. Comparison: Without NumPy vs. With NumPy

Suppose the marks of five students in Python are:

[85, 92, 65, 78, 55]

Without NumPy (Normal Python)

```
marks = [85, 92, 65, 78, 55]

total = 0
for mark in marks:
   total = total + mark

average = total / len(marks)
highest = max(marks)
lowest = min(marks)

above_80 = []
for mark in marks:
   if mark > 80:
      above_80.append(mark)

print("Total:", total)
print("Average:", average)
print("Highest:", highest)
print("Lowest:", lowest)
print("Above 80:", above_80)
```

We use loops to calculate the total and filter marks, while Python's built-in functions handle the maximum and minimum.

With NumPy

```
import numpy as np

marks = np.array([85, 92, 65, 78, 55])

print("Total:", np.sum(marks))
print("Average:", np.mean(marks))
print("Highest:", np.max(marks))
print("Lowest:", np.min(marks))
print("Above 80:", marks[marks > 80])
```

NumPy provides direct functions for calculations and lets us filter an entire array using a condition.

3. How NumPy Simplifies the Work

Operation	**Without NumPy**	**With NumPy**
Total marks	Loop through each mark and add it	np.sum(marks)
Average marks	Calculate total and divide by count	np.mean(marks)
Highest mark	max(marks)	np.max(marks)
Lowest mark	min(marks)	np.min(marks)
Filter marks above 80	Loop and append matching values	marks[marks > 80]
Add 5 bonus marks to every student	Loop and update each mark	marks + 5

For example, marks + 5 adds five to every element of a NumPy array in one operation. With a normal Python list, marks + 5 produces a TypeError; we would need a loop or a list comprehension.

4. Expected Results

Both approaches produce the same results for the calculations above.

Result	**Value**
Total marks	375
Average marks	75.0
Highest mark	92
Lowest mark	55
Marks above 80	85, 92

5. Conclusion

NumPy does not make every Python operation shorter, because normal Python already has useful built-in functions. Its main advantage is that it simplifies numerical calculations and operations across entire arrays, particularly when working with large datasets and multidimensional data.

For 5 students, the difference may be small. For thousands or millions of records, NumPy's vectorized operations can make numerical processing more convenient and often faster than equivalent Python loops.